

BOOK 11 — SPRING CORE

IoC, Dependency Injection & the Spring Container Internals (Telugu + English)

Series: Java Interview + Development Mastery Series **Book Number:** 11 of 24 **Spring Versions**

Covered: Spring Framework 5.x/6.x concepts (annotation-based configuration as the primary focus, with XML configuration covered for historical/legacy context) **Prerequisites:** Book 02 (SOLID, especially Dependency Inversion), Book 06 (Generics — Spring's type-based injection relies on them), Book 10 (layered architecture — this book formalizes it with DI) **Next Book:**

12_Spring_Boot.md

How to Use This Book / ఈ పుస్తకాన్ని ఎలా ఉపయోగించాలి

Telugu: Book 10 చివర్లో మనం `OrderController / OrderService / OrderRepository` ని `main()` లో manual గా wire చేశాము. ఈ పుస్తకం లో, Spring Framework ఆ wiring ని **automatic** గా ఎలా చేస్తుందో నేర్చుకుంటాము — Dependency Injection, IoC Container, Bean lifecycle. Spring "మాయ" కాదు — ఇది Book 02's Dependency Inversion Principle ని ఒక framework గా systematically అమలు చేయడమే.

English: At the end of Book 10, we manually wired

`OrderController / OrderService / OrderRepository` in `main()`. This book teaches how Spring Framework automates that wiring — Dependency Injection, the IoC Container, Bean lifecycle. Spring isn't magic — it's Book 02's Dependency Inversion Principle, systematically implemented as a framework.

Learning Objectives

1. Explain Inversion of Control (IoC) and Dependency Injection (DI), and why they matter.
 2. Understand the `ApplicationContext` and how beans are configured (Java config, component scanning, XML for context).
 3. Understand the full bean lifecycle and bean scopes.
 4. Use stereotype annotations (`@Component` / `@Service` / `@Repository` / `@Controller`) and component scanning correctly.
 5. Master `@Autowired` and the three injection types, plus `@Qualifier` / `@Primary` .
 6. Diagnose and resolve circular dependencies.
 7. Use `@Configuration` , `@Bean` , `@Value` , and `@Profile` for externalized, environment-specific configuration.
 8. Understand AOP fundamentals — proxies and cross-cutting concerns.
 9. Recognize the classic design patterns Spring uses internally.
-

Table of Contents

Ch.	Title
1	IoC & Dependency Injection — The Core Idea
2	ApplicationContext & Bean Configuration
3	Bean Lifecycle & Scopes
4	Stereotype Annotations & Component Scanning
5	@Autowired — Injection Types & Qualifiers
6	Circular Dependencies & Bean Wiring Deep Dive
7	Configuration & Profiles
8	AOP Fundamentals
9	Design Patterns Inside Spring
10	Mini Project — Plain Spring DI Application
—	Final Revision Notes, Cheat Sheet, Interview Bank, Revision Plans, Mastery Checklist

CHAPTER 1 — IoC & Dependency Injection: The Core Idea

Telugu Explanation

Inversion of Control (IoC) అంటే — ఎవరు object creation/wiring control చేస్తారో అనేది **మారడం** — normally మీ code `new SomeClass()` చేస్తుంది (control మీ దగ్గరే), కానీ IoC లో ఒక **container/framework** ఆ responsibility తీసుకుంటుంది. **Dependency Injection (DI)** IoC ని implement చేసే ఒక specific technique — ఒక class తన dependencies ని తనే create చేయకుండా, **బయటి నుండి ఇవ్వబడతాయి** (injected).

Professional English Explanation

Inversion of Control (IoC) means the responsibility for creating and wiring objects **moves away from your code** — normally your code calls `new SomeClass()` directly (you control creation), but under IoC, a **container/framework** takes over that responsibility. **Dependency Injection (DI)** is the specific technique implementing IoC — a class doesn't create its own dependencies; they're **supplied from outside** (injected).

Java Code — Before and After DI

```
// WITHOUT DI: OrderService creates its own dependency - tightly coupled (Book 02, Ch.13's DIP violation)
class OrderServiceWithoutDI {
    private final EmailNotifier notifier = new EmailNotifier();    // hardcoded concrete dependency
    void placeOrder() {
        System.out.println("Order placed.");
        notifier.notify("Order confirmation");
    }
}

class EmailNotifier {
    void notify(String message) { System.out.println("Email sent: " + message); }
}

// WITH DI: OrderService receives its dependency from OUTSIDE - loosely coupled
interface Notifier { void notify(String message); }

class EmailNotifierImpl implements Notifier {
    @Override public void notify(String message) { System.out.println("Email sent: " + message); }
}

class SmsNotifierImpl implements Notifier {
    @Override public void notify(String message) { System.out.println("SMS sent: " + message); }
}

class OrderServiceWithDI {
    private final Notifier notifier;    // depends on an ABSTRACTION (Book 02, DIP)
    OrderServiceWithDI(Notifier notifier) { this.notifier = notifier; } // injected via constructor - DI in its purest form
    void placeOrder() {
        System.out.println("Order placed.");
        notifier.notify("Order confirmation");
    }
}

public class IoCDependencyInjectionDemo {
    public static void main(String[] args) {
        // WITHOUT DI: OrderServiceWithoutDI is permanently stuck with EmailNotifier
        new OrderServiceWithoutDI().placeOrder();

        // WITH DI: the SAME OrderServiceWithDI class works with ANY Notifier implementation
        OrderServiceWithDI emailBased = new OrderServiceWithDI(new EmailNotifierImpl());
        OrderServiceWithDI smsBased = new OrderServiceWithDI(new SmsNotifierImpl());
        emailBased.placeOrder();
        smsBased.placeOrder();

        // This manual wiring (creating dependencies, passing them in) is EXACTLY what
        // Spring's IoC Container automates for you, at application-wide scale (Ch.2).
    }
}
```

Output

```
Order placed.  
Email sent: Order confirmation  
Order placed.  
Email sent: Order confirmation  
Order placed.  
SMS sent: Order confirmation
```

Internal Working

- This chapter's manual `new OrderServiceWithDI(new EmailNotifierImpl())` is **exactly** what Spring's `ApplicationContext` (Ch.2) does automatically for every bean in a real application — Spring reads your class's dependencies (via constructor parameters, or `@Autowired` fields/setters, Ch.5), figures out which beans satisfy them, creates everything in the correct order, and wires it all together — at the scale of potentially hundreds of interdependent objects, which would be extremely tedious and error-prone to wire by hand in a large application's `main()` method.
- The core enabling insight is Book 02, Ch.13's Dependency Inversion Principle: `OrderServiceWithDI` depends on the `Notifier` **interface**, not a concrete class — this is precisely why it's substitutable (email, SMS, or a test mock, Book 15) without ever touching `OrderServiceWithDI`'s own code; DI doesn't work without this abstraction-based design already in place.
- "Inversion" specifically refers to **who calls whom**: in traditional procedural/library code, your code calls into a library; under IoC, the framework calls into *your* code (constructing your objects, invoking your methods at the right time) — this is sometimes called the "Hollywood Principle": "don't call us, we'll call you."

Real-World Example

Telugu: Real Spring applications లో వందల కొద్దీ `@Service` / `@Repository` / `@Component` classes ఉంటాయి, ఒక్కటి మరొక్కదానికి dependency గా ఉంటాయి — వీటిని manual గా wire చేయడం అసాధ్యం అంతటి scale లో; Spring IoC Container ఇదే problem ని solve చేస్తుంది, ఈ chapter యొక్క manual wiring principle ని ఆటోమేటిక్ గా apply చేస్తూ. English: Real Spring applications have hundreds of `@Service` / `@Repository` / `@Component` classes, each depending on others — manually wiring all of them in `main()` at that scale would be impractical and error-prone; the Spring IoC Container solves exactly this problem, automatically applying this chapter's manual-wiring principle at scale.

Interview Answer

"Inversion of Control means object creation/wiring responsibility moves from your code to a container/framework. Dependency Injection is the specific technique implementing it — a class receives its dependencies from outside (via constructor, setter, or field) rather than creating them itself. This only works because of Dependency Inversion (Book 02) — classes depend on abstractions/interfaces, making them substitutable. Spring's `ApplicationContext` automates this wiring at application scale."

Deep Interview Answer

"IoC is sometimes summarized as the Hollywood Principle — 'don't call us, we'll call you' — inverting the traditional direction of control from your code actively creating/calling dependencies, to a framework actively constructing your objects and calling your methods. This inversion is what enables framework-provided cross-cutting behavior (transaction management, Book 09; security, Book 14; AOP, Ch.8) to be woven in around your business logic without your code needing to explicitly participate in or even be aware of that machinery — the framework controls the object lifecycle, so it can transparently intercept and augment it."

Cross Questions

- Q: Is DI the only way to implement IoC? → A: No — IoC is a broader principle; DI is the most common technique in modern frameworks, but Service Locator (a class actively asking a registry for its dependencies, rather than receiving them) is another, older IoC technique with different trade-offs.
- Q: Why can't DI work without Dependency Inversion (interfaces/abstractions) already in place? → A: If a class hardcodes a concrete dependency (`new EmailNotifierImpl()` directly inside it), there's nothing to "inject" — DI's value specifically comes from substitutability, which requires depending on an abstraction the injected implementation satisfies.
- Q: What is the "Hollywood Principle," and how does it relate to IoC? → A: "Don't call us, we'll call you" — describing how, under IoC, the framework calls into your code (constructing your beans, invoking lifecycle methods) rather than your code driving the framework, which is the essence of the "inversion."

Coding Exercise

L1: Refactor a tightly-coupled class (hardcoded concrete dependency) into a DI-friendly design using constructor injection. **L2:** Instantiate the same DI-friendly class with 2 different implementations of its dependency interface, without modifying the class itself. **L3:** Identify 3 places in your own past code where a hardcoded `new ConcreteClass()` inside a class could have been replaced with injected dependency. **L4 (Interview):** Explain IoC and DI, and why DI requires Dependency Inversion to already be in place. **L5 (Senior):** Explain the Hollywood Principle and connect it to how a framework can provide cross-cutting behavior (like transactions or security) around your business logic. **L6 (Mastery):** Explain, from memory, why manually wiring hundreds of interdependent objects in `main()` becomes impractical, motivating the need for an IoC container.

CHAPTER 2 — ApplicationContext & Bean Configuration

Telugu Explanation

ApplicationContext Spring యొక్క **IoC Container** — beans (Spring-managed objects) create చేసి, wire చేసి, manage చేస్తుంది. Beans ని configure చేసే మూడు పద్ధతులు: **Java-based configuration** (`@Configuration` + `@Bean` methods, modern default), **Component scanning** (`@Component` / `@Service` /etc. annotations, Ch.4 లో వివరంగా), **XML configuration** (legacy, ఇప్పటికీ కొన్ని old codebases లో కనిపిస్తుంది).

Professional English Explanation

ApplicationContext is Spring's **IoC Container** — it creates, wires, and manages beans (Spring-managed objects). Three ways to configure beans: **Java-based configuration** (`@Configuration` + `@Bean` methods, the modern default), **Component scanning** (`@Component` / `@Service` /etc. annotations, detailed in Ch.4), and **XML configuration** (legacy, still found in some older codebases).

Java Code

```
import org.springframework.context.annotation.*;
import org.springframework.context.ApplicationContext;

interface Notifier { void notify(String message); }

class EmailNotifierImpl implements Notifier {
    @Override public void notify(String message) { System.out.println("Email sent: " +
message); }
}

class OrderService {
    private final Notifier notifier;
    OrderService(Notifier notifier) { this.notifier = notifier; }
    void placeOrder() { notifier.notify("Order confirmed"); System.out.println("OrderService: "
+ this); }
}

@Configuration // marks this class as a source of bean
definitions
class AppConfig {

    @Bean // each @Bean method's return value
becomes a Spring-managed bean
    Notifier notifier() {
        return new EmailNotifierImpl();
    }

    @Bean
    OrderService orderService(Notifier notifier) { // Spring INJECTS the 'notifier'
bean automatically as a parameter
        return new OrderService(notifier);
    }
}

public class ApplicationContextDemo {
    public static void main(String[] args) {
        ApplicationContext context = new AnnotationConfigApplicationContext(AppConfig.class);

        OrderService orderService = context.getBean(OrderService.class); // retrieve a
fully-wired bean
        orderService.placeOrder();

        // Beans are managed - retrieving the SAME bean type twice returns the SAME instance
(default scope, Ch.3)
        OrderService again = context.getBean(OrderService.class);
        System.out.println("Same instance? " + (orderService == again));

        System.out.println("All bean names: " +
java.util.Arrays.toString(context.getBeanDefinitionNames()));
    }
}
```


Output

```
OrderService: OrderService@1a2b3c4d
Order placed... wait, corrected below
```

Corrected Output

```
Email sent: Order confirmed
OrderService: OrderService@1a2b3c4d
Same instance? true
All bean names: [appConfig, notifier, orderService]
```

Internal Working

- `@Configuration` classes are themselves processed specially by Spring — calling a `@Bean` method (like `orderService(notifier())`) doesn't naively re-invoke `notifier()` and create a second `Notifier` instance each time it's referenced; Spring uses **CGLIB proxying** (Ch.9's preview of Spring's internal patterns) to intercept `@Bean` method calls within a `@Configuration` class and return the **already-created, container-managed instance** instead — this is precisely what guarantees singleton behavior (Ch.3) even when one `@Bean` method calls another directly in Java code.
- `context.getBean(OrderService.class)` performs a **type-based lookup** — Spring resolves which registered bean(s) match that type; if a `@Bean` method's parameter type (`Notifier notifier`) matches exactly one bean in the container, Spring **automatically injects it** as an argument when calling that method — this parameter-based auto-wiring inside `@Bean` methods is functionally equivalent to `@Autowired` (Ch.5), just expressed differently.
- The container builds beans in **dependency order** — since `orderService()` depends on the `notifier` bean, Spring ensures `notifier()` is invoked (and its result available) before `orderService()` needs it, regardless of the declaration order of the `@Bean` methods in the `@Configuration` class.

Real-World Example

Telugu: Spring Boot applications (Book 12) `@SpringBootApplication` annotation లోపల `@Configuration` కూడా include అయి ఉంటుంది — `ApplicationContext` startup అయినప్పుడు, మీ app యొక్క అన్ని `@Component` / `@Service` / `@Bean` definitions ని scan చేసి, dependency graph build చేసి, correct order లో create చేస్తుంది. English: A Spring Boot application's `@SpringBootApplication` annotation (Book 12) itself includes `@Configuration` — when the `ApplicationContext` starts up, it scans all your `@Component` / `@Service` / `@Bean` definitions, builds the full dependency graph, and creates everything in the correct order — this chapter's manual `AnnotationConfigApplicationContext` example is exactly what happens under the hood at Spring Boot startup, just automated further.

Interview Answer

"`ApplicationContext` is Spring's IoC container, responsible for creating, wiring, and managing beans. Beans can be defined via `@Configuration` classes with `@Bean` methods, via component

scanning (`@Component` / `@Service` /etc., Ch.4), or legacy XML. `@Configuration` classes are specially processed via CGLIB proxying so that calling one `@Bean` method from another returns the already-managed singleton instance rather than creating duplicates — this is what guarantees consistent singleton behavior across the whole bean graph."

Cross Questions

- Q: If `orderService()` calls `notifier()` directly in Java code within a `@Configuration` class, does that create a second `Notifier` instance? → A: No — Spring's CGLIB-based `@Configuration` proxying intercepts that call and returns the already-managed singleton bean instance instead of naively re-executing the method body.
- Q: How does Spring know to inject the `notifier` bean into the `orderService(Notifier notifier)` method automatically? → A: By type-matching the parameter type (`Notifier`) against registered beans in the container — if exactly one bean of that type exists, it's injected automatically.
- Q: Does calling `context.getBean(OrderService.class)` twice return the same object? → A: By default, yes — the default bean scope is singleton (one instance per container, Ch.3), so repeated lookups of the same bean type/name return the identical instance.

Tricky Questions

- Q: What happens if `AppConfig`'s `@Bean` methods aren't processed via a CGLIB proxy (e.g., in "lite" mode, `@Configuration(proxyBeanMethods = false)`)? → A: Calling one `@Bean` method directly from another WOULD then naively re-execute the method body, potentially creating a duplicate, non-container-managed instance — `proxyBeanMethods = false` is a legitimate performance optimization for configuration classes that don't rely on this inter-bean-method-call singleton guarantee, but it changes this specific behavior and must be used deliberately.
- Q: Does the `@Configuration` class itself become a bean in the container? → A: Yes — as shown in the output (`appConfig` appears in `getBeanDefinitionNames()`), the configuration class is itself registered as a bean.

Coding Exercise

L1: Create an `AppConfig` class with 2 `@Bean` methods where one depends on the other, and retrieve the wired bean via `ApplicationContext`. **L2:** Verify singleton behavior by retrieving the same bean type twice and comparing references. **L3:** Print all registered bean names via `getBeanDefinitionNames()` and explain what each represents. **L4 (Interview):** Explain how `@Configuration`'s CGLIB proxying guarantees singleton behavior for inter-bean-method calls. **L5 (Senior):** Explain the trade-off of `@Configuration(proxyBeanMethods = false)` and when it might be a deliberate choice. **L6 (Mastery):** Explain, from memory, how Spring resolves a `@Bean` method's parameters (like `notifier` in `orderService(Notifier notifier)`) to existing beans in the container.

CHAPTER 3 — Bean Lifecycle & Scopes

Telugu Explanation

Bean lifecycle: **Instantiation** → **Dependency injection** (properties set) → **@PostConstruct** (initialization callback) → **bean ready to use** → (container shutdown) → **@PreDestroy** (cleanup callback). Bean **scope** ఎన్ని instances create అవుతాయో నిర్ణయిస్తుంది: **singleton** (default — ఒక్క instance per container), **prototype** (ప్రతి `getBean()` call కి కొత్త instance), **request/session** (web-specific, Book 12).

Professional English Explanation

Bean lifecycle: **Instantiation** → **Dependency injection** (properties/dependencies set) → **@PostConstruct** (initialization callback) → **bean ready for use** → (container shutdown) → **@PreDestroy** (cleanup callback). Bean **scope** determines how many instances are created: **singleton** (default — exactly one instance per container), **prototype** (a new instance on every `getBean()` call), **request/session** (web-specific scopes, Book 12).

Java Code

```
import org.springframework.context.annotation.*;
import org.springframework.beans.factory.annotation.Value;
import jakarta.annotation.*;
import org.springframework.context.ConfigurableApplicationContext;

class DatabaseConnectionPool {
    private boolean connected = false;

    @PostConstruct // runs AFTER dependency
    injection, BEFORE the bean is used
    void init() {
        connected = true;
        System.out.println("DatabaseConnectionPool: connection pool initialized (like Book 09, Ch.7)");
    }

    void query() {
        if (!connected) throw new IllegalStateException("Not initialized!");
        System.out.println("Executing query...");
    }

    @PreDestroy // runs when the container
    shuts down, BEFORE bean destruction
    void cleanup() {
        connected = false;
        System.out.println("DatabaseConnectionPool: connections closed cleanly");
    }
}

@Scope("prototype") // new instance EVERY time it's
requested
class RequestScopedTask {
    private static int counter = 0;
    private final int id;
    RequestScopedTask() { id = ++counter; System.out.println("New RequestScopedTask created: id=" + id); }
    int getId() { return id; }
}

@Configuration
class LifecycleConfig {
    @Bean DatabaseConnectionPool pool() { return new DatabaseConnectionPool(); }
    @Bean RequestScopedTask requestScopedTask() { return new RequestScopedTask(); } //
    scope annotation on class applies
}

public class BeanLifecycleScopesDemo {
    public static void main(String[] args) {
        ConfigurableApplicationContext context = new
        AnnotationConfigApplicationContext(LifecycleConfig.class);

        DatabaseConnectionPool pool = context.getBean(DatabaseConnectionPool.class);
        pool.query();

        // Singleton (default) - same instance every time
        DatabaseConnectionPool poolAgain = context.getBean(DatabaseConnectionPool.class);
        System.out.println("Pool same instance? " + (pool == poolAgain));
    }
}
```

```

// Prototype - DIFFERENT instance every time
RequestScopedTask task1 = context.getBean(RequestScopedTask.class);
RequestScopedTask task2 = context.getBean(RequestScopedTask.class);
System.out.println("Task same instance? " + (task1 == task2) + " (ids: " +
task1.getId() + ", " + task2.getId() + ")");

context.close(); // triggers @PreDestroy on
singleton beans
}
}

```

Output

```

DatabaseConnectionPool: connection pool initialized (like Book 09, Ch.7)
Executing query...
Pool same instance? true
New RequestScopedTask created: id=1
New RequestScopedTask created: id=2
Task same instance? false (ids: 1, 2)
DatabaseConnectionPool: connections closed cleanly

```

Diagram — Full Bean Lifecycle

```

1. Container reads bean definition (@Bean method, @Component class, etc.)
2. INSTANTIATION - constructor called
3. DEPENDENCY INJECTION - constructor args / @Autowired fields/setters populated (Ch.5)
4. Aware interfaces called if implemented (BeanNameAware, ApplicationContextAware, etc.)
5. @PostConstruct method(s) called - bean is now FULLY initialized and ready
   |
   [ BEAN IS ACTIVE AND IN USE ]
   |
6. Container shutdown initiated (context.close())
7. @PreDestroy method(s) called - cleanup, resource release
8. Bean eligible for garbage collection (Book 03)

```

Internal Working

- **@PostConstruct exists specifically because dependency injection isn't complete at constructor time** for setter/field injection (Ch.5) — a constructor can't safely use injected fields that are set *after* construction via setters, so **@PostConstruct** guarantees all injection (regardless of type) has fully completed before initialization logic runs; this is precisely why the connection-pool `init()` method is placed there rather than in the constructor.
- **Prototype-scoped beans are NOT managed through their full lifecycle by the container** — Spring creates and initializes them (including **@PostConstruct**) but, critically, does **not** call **@PreDestroy** or otherwise track them for cleanup after handing them off — the calling code becomes responsible for that bean's lifecycle/cleanup from that point on, a genuinely important and sometimes-surprising distinction from singleton beans.
- `context.close()` is what triggers **@PreDestroy** callbacks on **singleton** beans (not prototypes, per above) — this is exactly why singleton beans are the correct place for resources needing guaranteed cleanup (Book 09's connection pools, Book 08's thread pools) tied to the whole application's lifecycle, while prototype beans suit short-lived, per-use objects.

Real-World Example

Telugu: Real Spring Boot applications లో connection pools, thread pools, cache managers — అన్నీ singleton scope beans గా design అవుతాయి, `@PostConstruct` / `@PreDestroy` వాడి, application startup/shutdown తో సరిగ్గా sync అయ్యేలా. Prototype scope అరుదుగా వాడతారు, specific "per-use, stateful, short-lived object" scenarios కి మాత్రమే. English: Real Spring Boot applications design connection pools, thread pools, and cache managers as singleton-scoped beans using `@PostConstruct` / `@PreDestroy` to correctly sync with application startup/shutdown; prototype scope is used far more rarely, only for genuinely per-use, stateful, short-lived object scenarios.

Interview Answer

"A bean's lifecycle is: instantiation, dependency injection, `@PostConstruct` (initialization, guaranteed to run after ALL injection completes, regardless of injection type), active use, and `@PreDestroy` (cleanup) on container shutdown. Scope determines instance count — singleton (default, one per container, fully lifecycle-managed including `@PreDestroy`) versus prototype (a new instance per request, NOT tracked for destroy-time cleanup by the container after creation)."

Deep Interview Answer

"The distinction that trips people up most is that prototype beans are only *partially* lifecycle-managed — Spring creates, injects, and initializes them (`@PostConstruct` runs), but once handed off, the container loses track of them for destruction purposes; if a prototype bean holds a resource needing explicit cleanup, the *creating* code must handle that itself (or use a `DisposableBean`-aware custom destruction mechanism), since `@PreDestroy` will silently never fire. This is a genuine, documented gotcha that has caused real resource leaks in production Spring applications when developers assumed prototype beans behave like singletons for cleanup purposes."

Cross Questions

- Q: Why can't initialization logic reliably go in the constructor for a bean using setter/field injection? → A: Because those dependencies aren't set yet at constructor-call time — only after construction, via the setter/field injection step — so `@PostConstruct` (guaranteed to run after ALL injection, any type) is the safe place for logic depending on injected dependencies.
- Q: Does `@PreDestroy` fire for prototype-scoped beans when the container shuts down? → A: No — the container does not track prototype beans for destruction after handing them off; only singleton beans get this guaranteed cleanup callback.
- Q: What is the default bean scope in Spring? → A: Singleton — exactly one shared instance per `ApplicationContext`.

Tricky Questions

- Q: If a singleton bean's constructor itself throws an exception, does `@PreDestroy` ever get called for it? → A: No — if construction (or the subsequent injection/ `@PostConstruct` initialization) fails, the bean never reaches the "fully initialized and active" state, so there's nothing valid to call `@PreDestroy` on; the container typically fails to start entirely in this case (a fail-fast startup behavior).

- Q: Is it safe to inject a prototype-scoped bean into a singleton bean via a normal field injection?
→ A: This is a well-known gotcha — since the singleton is only constructed/injected ONCE, it would receive and hold onto just ONE instance of the "prototype" bean forever, defeating the intended "new instance every time" semantics; the correct pattern for this scenario uses a `ObjectFactory / Provider` injection or method injection instead, specifically to get a fresh prototype instance on each actual use.

Coding Exercise

L1: Build a bean with `@PostConstruct / @PreDestroy` and observe both firing at the correct lifecycle points. **L2:** Compare singleton vs prototype scope by retrieving each bean type twice and comparing instance identity. **L3:** Reproduce the "prototype bean's `@PreDestroy` never fires" behavior and confirm it in your output. **L4 (Interview):** Explain the full bean lifecycle from instantiation to destruction. **L5 (Senior):** Explain the "singleton holding a stale prototype reference" gotcha and how `ObjectFactory / Provider` injection solves it. **L6 (Mastery):** Explain, from memory, why `@PostConstruct` exists as a separate step from the constructor, connecting it to the different injection types (Ch.5).

CHAPTER 4 — Stereotype Annotations & Component Scanning

Telugu Explanation

Stereotype annotations (`@Component` , `@Service` , `@Repository` , `@Controller` / `@RestController`) ఒక class ని Spring-managed bean గా mark చేస్తాయి, **layer-specific semantic meaning** తో (Book 10, Ch.9's layered architecture కి direct annotation mapping). **Component scanning** (`@ComponentScan`) Spring కి specific packages లో ఈ annotations ఉన్న classes ని automatic గా find చేసి register చేయమని చెప్తుంది — `@Bean` methods manual గా రాయాల్సిన అవసరం లేకుండా.

Professional English Explanation

Stereotype annotations (`@Component` , `@Service` , `@Repository` , `@Controller` / `@RestController`) mark a class as a Spring-managed bean, with **layer-specific semantic meaning** — a direct annotation-based mapping onto Book 10, Ch.9's layered architecture. **Component scanning** (`@ComponentScan`) tells Spring to automatically find and register classes carrying these annotations within specified packages, eliminating the need to manually write a `@Bean` method for every single class.

Java Code

```
import org.springframework.stereotype.*;
import org.springframework.context.annotation.*;
import org.springframework.context.ApplicationContext;

@Repository // data access layer (Book 09/10) - Spring
also adds exception translation here
class OrderRepository {
    java.util.List<String> findAllOrders() { return java.util.List.of("Order-1", "Order-2"); }
}

@Service // business logic layer (Book 10, Ch.9)
class OrderService {
    private final OrderRepository repository;
    OrderService(OrderRepository repository) { this.repository = repository; } //
    constructor injection - no @Autowired NEEDED
    java.util.List<String> getAllOrders() { return repository.findAllOrders(); }
}

@RestController // presentation/HTTP layer (Book 10, Ch.1-2)
- covered fully in Book 12
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) { this.service = service; }
    java.util.List<String> handleGetOrders() { return service.getAllOrders(); } //
    simulates a @GetMapping handler
}

@Component // generic bean - doesn't fit a specific
layer stereotype
class RequestIdGenerator {
    private final java.util.concurrent.atomic.AtomicLong counter = new
    java.util.concurrent.atomic.AtomicLong();
    String generate() { return "REQ-" + counter.incrementAndGet(); } // Book
    08 atomic usage
}

@Configuration
@ComponentScan(basePackages = "book11") // tells Spring WHERE to look for
@Component/@Service/etc.
class ScanningConfig {}

public class ComponentScanningDemo {
    public static void main(String[] args) {
        ApplicationContext context = new
        AnnotationConfigApplicationContext(ScanningConfig.class);

        OrderController controller = context.getBean(OrderController.class);
        System.out.println("Orders: " + controller.handleGetOrders());

        RequestIdGenerator generator = context.getBean(RequestIdGenerator.class);
        System.out.println("Generated ID: " + generator.generate());

        System.out.println("Registered beans: " +
        java.util.Arrays.toString(context.getBeanDefinitionNames()));
    }
}
```

Output

```
Orders: [Order-1, Order-2]
Generated ID: REQ-1
Registered beans: [scanningConfig, orderRepository, orderService, orderController,
requestIdGenerator]
```

Internal Working

- **All 4 stereotype annotations are functionally identical to `@Component`** at the container-registration level (they're all meta-annotated with `@Component` themselves) — the differences are purely **semantic** (documenting a class's architectural role) and, for `@Repository` specifically, one genuine functional addition: Spring wraps `@Repository`-annotated beans with automatic **exception translation** (converting a data-access-technology-specific exception, like a raw `SQLException`, Book 09, into a consistent Spring `DataAccessException` hierarchy) — the one case where the choice of stereotype annotation has real runtime behavioral consequence, not just documentation value.
- **Constructor injection needs no `@Autowired` annotation at all** in modern Spring (since Spring 4.3+) when a class has **exactly one constructor** — Spring automatically uses it for injection; `@Autowired` becomes necessary only when a class has multiple constructors and you must indicate which one Spring should use for injection (Ch.5 covers this fully).
- Component scanning works by Spring's classpath scanner examining `.class` files within the specified base package(s) at startup, looking for the stereotype annotations (or any custom annotation itself meta-annotated with `@Component`) — this is a genuine reflection-based classpath scan, which is why very large codebases sometimes narrow `@ComponentScan`'s base packages deliberately, to keep startup scanning time reasonable.

Real-World Example

Telugu: Book 10, Ch.9 లో manual గా build చేసిన `OrderController / OrderService / OrderRepository` layered structure — ఇక్కడ కేవలం stereotype annotations add చేయడం ద్వారా, పూర్తి wiring Spring కి అప్పగించబడింది. ఇదే annotation-driven architecture real Spring Boot applications (Book 12) లో default గా వాడతారు. English: Book 10, Ch.9's manually-built

`OrderController / OrderService / OrderRepository` layered structure — here, simply adding stereotype annotations hands the entire wiring responsibility to Spring. This exact annotation-driven architecture is the default in real Spring Boot applications (Book 12).

Interview Answer

"`@Component`, `@Service`, `@Repository`, and `@Controller / @RestController` all register a class as a Spring-managed bean — functionally identical except `@Repository`, which additionally enables automatic exception translation for data-access code. The differences are primarily semantic, documenting architectural role per Book 10's layered architecture. Component scanning (`@ComponentScan`) automatically discovers and registers annotated classes across specified packages, eliminating manual `@Bean` method boilerplate. Since Spring 4.3+, a class with exactly one constructor needs no explicit `@Autowired` at all."

Cross Questions

- Q: What's the one genuine functional difference between `@Repository` and plain `@Component`? → A: `@Repository` additionally enables Spring's automatic exception translation, converting technology-specific data-access exceptions into a consistent `DataAccessException` hierarchy.
- Q: Do you need `@Autowired` on a constructor if the class has only one constructor? → A: No, not since Spring 4.3+ — Spring automatically uses the sole constructor for injection; `@Autowired` is only required when disambiguating among multiple constructors.
- Q: How does component scanning find annotated classes? → A: Via a reflection-based classpath scan of the specified base package(s) at container startup, looking for classes carrying `@Component` or any annotation meta-annotated with it.

Tricky Questions

- Q: If `@ComponentScan`'s base package doesn't include a package containing an `@Service`-annotated class, what happens? → A: That class is simply never registered as a bean — no error at that point, but anything depending on it (needing it injected) will fail to start with a "no qualifying bean" error, since the container genuinely doesn't know it exists.
- Q: Can you create a custom stereotype annotation with your own semantic meaning? → A: Yes — any custom annotation meta-annotated with `@Component` is itself recognized by component scanning, a pattern used by frameworks/large codebases to create domain-specific stereotypes (e.g., a custom `@UseCase` annotation) while still participating fully in Spring's bean registration.

Coding Exercise

L1: Annotate a 3-layer class structure (`@Repository` / `@Service` / `@RestController`) and wire it via component scanning. **L2:** Remove `@Autowired` from a single-constructor class and confirm injection still works (Spring 4.3+ behavior). **L3:** Create a custom annotation meta-annotated with `@Component` and confirm component scanning discovers it. **L4 (Interview):** Explain the one functional difference between `@Repository` and `@Component`. **L5 (Senior):** Review a large codebase's `@ComponentScan` configuration scanning an overly broad base package — explain the startup-time cost and propose a narrower scanning strategy. **L6 (Mastery):** Explain, from memory, why all 4 stereotype annotations are "functionally identical to `@Component`" except for one specific case.

CHAPTER 5 — @Autowired: Injection Types & Qualifiers

Telugu Explanation

మూడు injection types: **Constructor injection** (recommended — immutable fields, `final` వాడొచ్చు, testability better), **Setter injection** (optional dependencies కి), **Field injection** (`@Autowired` నేరుగా field మీద — concise కానీ testing/immutability కి discouraged). ఒకే type కి multiple beans ఉంటే, `@Qualifier` (specific bean name specify చేయడానికి) లేదా `@Primary` (default choice mark చేయడానికి) వాడతారు.

Professional English Explanation

Three injection types: **Constructor injection** (recommended — enables `final` immutable fields, best testability, makes required dependencies explicit), **Setter injection** (suited to optional dependencies), **Field injection** (`@Autowired` directly on a field — concise but discouraged for testability/immutability reasons). When multiple beans of the same type exist, `@Qualifier` (specify a particular bean by name) or `@Primary` (mark a default choice) resolve the ambiguity.

Java Code

```
import org.springframework.beans.factory.annotation.*;
import org.springframework.context.annotation.*;
import org.springframework.stereotype.*;

interface PaymentGateway { void charge(double amount); }

@Component
@Primary // the DEFAULT choice when multiple
PaymentGateway beans exist
class StripeGateway implements PaymentGateway {
    @Override public void charge(double amount) { System.out.println("Charged $" + amount + "
via Stripe"); }
}

@Component("razorpayGateway") // explicit bean name for
@Qualifier to reference
class RazorpayGateway implements PaymentGateway {
    @Override public void charge(double amount) { System.out.println("Charged ₹" + amount + "
via Razorpay"); }
}

@Service
class ConstructorInjectedCheckoutService { // RECOMMENDED approach
    private final PaymentGateway gateway; // can be 'final' - only
possible with constructor injection

    ConstructorInjectedCheckoutService(@Qualifier("razorpayGateway") PaymentGateway gateway) {
        this.gateway = gateway; // explicitly choosing
Razorpay, not the @Primary Stripe
    }
    void checkout(double amount) { gateway.charge(amount); }
}

@Service
class FieldInjectedCheckoutService { // DISCOURAGED - shown for
comparison only
    @Autowired // uses @Primary (Stripe)
since no @Qualifier here
    private PaymentGateway gateway; // can't be 'final' - a
real limitation of field injection
    void checkout(double amount) { gateway.charge(amount); }
}

@Configuration
@ComponentScan(basePackages = "book11")
class InjectionConfig {}

public class AutowiredInjectionDemo {
    public static void main(String[] args) {
        var context = new AnnotationConfigApplicationContext(InjectionConfig.class);

        ConstructorInjectedCheckoutService qualified =
context.getBean(ConstructorInjectedCheckoutService.class);
        qualified.checkout(100.0); // Razorpay, per
@Qualifier

        FieldInjectedCheckoutService primaryBased =
context.getBean(FieldInjectedCheckoutService.class);
    }
}
```

```

        primaryBased.checkout(50.0); // Stripe, per
@Primary (no qualifier specified)
    }
}

```

Output

```

Charged ₹100.0 via Razorpay
Charged $50.0 via Stripe

```

Comparison Table

Injection Type	Immutability (final)	Testability	Required-ness Clear?	Recommendation
Constructor	✓ Yes	✓ Best (easy to construct with mocks in tests, Book 15)	✓ Yes (can't construct without it)	Recommended default
Setter	✗ No	⚠ OK (needs a setter call in tests)	✗ No (object can exist in a partially-configured state)	Optional dependencies only
Field	✗ No	✗ Worst (requires reflection/Spring context to set in tests)	✗ No	Discouraged

Internal Working

- Constructor injection's testability advantage is concrete, not just stylistic: a unit test (Book 15) can simply call `new ConstructorInjectedCheckoutService(mockGateway)` directly — no Spring context needed at all for the test — while field injection requires either reflection tricks or actually starting a (test) Spring context just to populate the field, adding real friction and slowness to what should be a fast, isolated unit test.
- `@Qualifier` and `@Primary` solve the **same underlying problem** (ambiguous injection when multiple beans of one type exist) from two different angles: `@Primary` declares a bean's own **default candidacy** (used whenever no more specific qualifier is given), while `@Qualifier` is used at the **injection point** to explicitly override that default for one specific dependency — both can coexist, as the demo shows (Stripe is `@Primary`, but the Razorpay-qualified injection point explicitly overrides it).
- If multiple beans of the same type exist and **neither** `@Primary` nor `@Qualifier` disambiguates a given injection point, Spring throws `NoUniqueBeanDefinitionException` at startup — a fail-fast behavior (application won't even start) rather than an unpredictable runtime choice, which is a deliberate, valuable safety property.

Real-World Example

Telugu: Multi-provider payment systems (Stripe, Razorpay, PayPal) లో, `@Primary` default gateway ని mark చేసి, specific checkout flows `@Qualifier` వాడి explicit గా వేరే gateway select చేసుకోగలవు — ఇది Book 02, Ch.13's DIP ని Spring annotations ద్వారా production లో ఎలా వాడతారో చూపిస్తుంది. English:

Multi-provider payment systems (Stripe, Razorpay, PayPal) mark a default gateway with `@Primary` while specific checkout flows use `@Qualifier` to explicitly select a different one — a direct, practical demonstration of how Book 02, Ch.13's Dependency Inversion Principle is applied in production via Spring's annotation-driven injection.

Interview Answer

"Constructor injection is the recommended default — it enables `final` immutable fields, makes required dependencies explicit and unconstructable without them, and is easiest to unit test since no Spring context is needed. Setter injection suits optional dependencies; field injection is discouraged for testability and immutability reasons. When multiple beans of the same type exist, `@Primary` marks a default candidate and `@Qualifier` explicitly selects a specific bean at an injection point, overriding the default when both are present."

Cross Questions

- Q: Why is constructor injection considered best for testability? → A: A unit test can directly construct the class with mock dependencies via `new`, with no Spring context needed at all — field injection requires reflection or an actual (test) Spring context just to populate fields.
- Q: What happens if two beans implement the same interface and neither `@Primary` nor `@Qualifier` is used at an ambiguous injection point? → A: `NoUniqueBeanDefinitionException` at application startup — a deliberate fail-fast behavior rather than an unpredictable runtime guess.
- Q: Can `@Primary` and `@Qualifier` be used together, and if so, which wins? → A: Yes — `@Qualifier` at a specific injection point overrides `@Primary`'s default for that one injection point specifically, while other unqualified injection points of that type still receive the `@Primary` bean.

Tricky Questions

- Q: Since Spring 4.3+, is `@Autowired` ever still required on a constructor? → A: Yes — specifically when a class has multiple constructors, Spring needs `@Autowired` on exactly one of them to know which to use for bean creation; with only one constructor, it's implied automatically.
- Q: Is field injection ever a defensible choice in real production code? → A: It's broadly discouraged, but some teams accept it for very simple, rarely-tested utility/configuration classes where the testability cost is judged minimal — this is a real, if controversial, team-style decision, not a hard technical prohibition.

Coding Exercise

L1: Build 2 implementations of an interface, mark one `@Primary`, and inject both via constructor injection using `@Qualifier` for the non-primary one. **L2:** Convert a field-injected class to constructor injection, and write a simple `new ClassName(mockDependency)` test-style instantiation demonstrating the testability improvement. **L3:** Deliberately create an ambiguous injection scenario (2 beans, no `@Primary` / `@Qualifier`) and observe `NoUniqueBeanDefinitionException`. **L4**

(Interview): Explain the three injection types and why constructor injection is generally recommended. **L5 (Senior):** Review a codebase using field injection throughout — propose a migration plan to constructor injection and the testability benefit it unlocks. **L6 (Mastery):** Explain,

from memory, the difference between `@Primary` and `@Qualifier`, and how they interact when both are present.

CHAPTER 6 — Circular Dependencies & Bean Wiring Deep Dive

Telugu Explanation

Circular dependency అంటే Bean A, Bean B మేరీద depend అవుతుంది, Bean B కూడా Bean A మేరీద depend అవుతుంది — ఒక "chicken-and-egg" problem, ఎవరిని ముందు create చేయాలో. **Constructor injection తో circular dependencies compile-time/startup-time error** గా fail అవుతాయి (Spring దీన్ని detect చేసి startup లోనే fail చేస్తుంది) — ఇది actually **మంచిది**, ఎందుకంటే circular dependency సాధారణంగా ఒక design smell.

Professional English Explanation

A circular dependency means Bean A depends on Bean B, and Bean B also depends on Bean A — a "chicken-and-egg" problem of which to create first. With **constructor injection, circular dependencies fail fast at startup** (Spring detects and fails immediately) — this is actually **beneficial**, since a circular dependency is generally a design smell worth fixing, not working around.

Java Code

```
import org.springframework.stereotype.*;
import org.springframework.context.annotation.*;
import org.springframework.beans.factory.annotation.Autowired;

// PROBLEM: circular dependency via constructor injection - FAILS FAST at startup
@Service
class ServiceA_Broken {
    private final ServiceB_Broken serviceB;
    ServiceA_Broken(ServiceB_Broken serviceB) { this.serviceB = serviceB; } // needs B
    to be constructed FIRST...
}
@Service
class ServiceB_Broken {
    private final ServiceA_Broken serviceA;
    ServiceB_Broken(ServiceA_Broken serviceA) { this.serviceA = serviceA; } // ...but B
    needs A to be constructed first!
}
// Running this configuration throws: BeanCurrentlyInCreationException at startup - Spring
REFUSES to guess

// THE RIGHT FIX: redesign to break the cycle - extract shared logic into a third class
@Component
class SharedLogic {
    void doCommonWork() { System.out.println("Shared logic executed"); }
}
@Service
class ServiceA_Fixed {
    private final SharedLogic sharedLogic;
    ServiceA_Fixed(SharedLogic sharedLogic) { this.sharedLogic = sharedLogic; }
    void doWork() { sharedLogic.doCommonWork(); System.out.println("ServiceA_Fixed working"); }
}
@Service
class ServiceB_Fixed {
    private final SharedLogic sharedLogic;
    ServiceB_Fixed(SharedLogic sharedLogic) { this.sharedLogic = sharedLogic; }
    void doWork() { sharedLogic.doCommonWork(); System.out.println("ServiceB_Fixed working"); }
}

@Configuration
@ComponentScan(basePackages = "book11.fixed") // ONLY scans the fixed
versions, not the broken ones
class CircularDependencyFixedConfig {}

public class CircularDependencyDemo {
    public static void main(String[] args) {
        var context = new
AnnotationConfigApplicationContext(CircularDependencyFixedConfig.class);
        context.getBean(ServiceA_Fixed.class).doWork();
        context.getBean(ServiceB_Fixed.class).doWork();

        System.out.println("""

                If ServiceA_Broken/ServiceB_Broken were scanned instead, Spring would throw:
                BeanCurrentlyInCreationException: Error creating bean with name
'serviceA_Broken':
                Requested bean is currently in creation: Is there an unresolvable circular
reference?

                This is a FEATURE, not a limitation - it forces you to fix the underlying

```

```

design smell
    rather than silently working around it.
    """);
}
}

```

Output

```

Shared logic executed
ServiceA_Fixed working
Shared logic executed
ServiceB_Fixed working

If ServiceA_Broken/ServiceB_Broken were scanned instead, Spring would throw:
  BeanCurrentlyInCreationException: Error creating bean with name 'serviceA_Broken':
    Requested bean is currently in creation: Is there an unresolvable circular reference?
This is a FEATURE, not a limitation - it forces you to fix the underlying design smell
rather than silently working around it.

```

Internal Working

- With **constructor injection**, Spring must have a bean fully constructed (all its constructor arguments resolved) before it can be injected anywhere else — a cycle (A needs B needs A) has no valid starting point, so Spring correctly detects this is unresolvable and fails fast with `BeanCurrentlyInCreationException`, rather than silently producing a broken or `null` - containing object graph.
- **Setter/field injection can technically "work around" a circular dependency** — Spring can construct A (with an empty/uninitialized B reference), then construct B (injecting the partially-constructed A), then go back and complete A's setter injection with the now-available B — but this is widely considered a genuine anti-pattern: it works only by accident of Spring's internal bean-creation ordering, produces objects other code might observe in a partially-initialized state during startup, and — most importantly — the *underlying design smell* (two classes needing each other) remains unaddressed, just hidden.
- The **correct fix**, almost always, is redesigning to break the cycle: extracting shared logic into a third class both depend on (as shown), or reconsidering whether the two responsibilities genuinely belong together (perhaps merging them into one class if they're truly inseparable) — a direct, practical application of Book 02, Ch.14's coupling/cohesion principles.

Real-World Example

Telugu: Large enterprise codebases లో circular dependencies తరచుగా poor separation of concerns వల్ల వస్తాయి — `UserService` needs `OrderService` needs `UserService` వంటివి. Correct fix ఎప్పుడూ "workaround" (setter injection తో force చేయడం) కాదు, actual **architecture refactor**. English: In large enterprise codebases, circular dependencies commonly arise from poor separation of concerns — `UserService` needing `OrderService` needing `UserService` back. The correct fix is never a "workaround" (forcing it via setter injection) — it's a genuine architectural refactor, exactly the discipline this chapter emphasizes.

Interview Answer

"A circular dependency means two beans depend on each other, creating an unresolvable ordering problem for construction. With constructor injection, Spring fails fast at startup with

`BeanCurrentlyInCreationException` — a deliberate, beneficial safety property that forces fixing the underlying design smell rather than silently working around it. Setter/field injection can technically 'resolve' the cycle through partial construction, but this is an anti-pattern that hides rather than fixes the real problem. The correct fix is redesigning — typically extracting shared logic into a third class both depend on."

Cross Questions

- Q: Why does constructor injection fail fast on circular dependencies while setter injection can "work around" them? → A: Constructor injection requires all dependencies fully resolved before the object exists at all; setter injection allows an object to exist first (uninitialized) and be populated afterward, which technically permits breaking the cycle at the cost of objects existing in a temporarily incomplete state during startup.
- Q: Is "working around" a circular dependency via setter injection a good practice? → A: No — it's widely considered an anti-pattern, since it hides rather than fixes the actual design problem (excessive coupling between the two classes), and relies on Spring's internal ordering behavior rather than a clean, understandable design.
- Q: What's the standard, correct fix for a genuine circular dependency? → A: Redesign to break the cycle — commonly by extracting the shared/common logic both classes need into a third class they both depend on, rather than depending on each other directly.

Tricky Questions

- Q: Can a circular dependency ever be a legitimate, unavoidable design (not a smell to fix)? → A: Extremely rarely, and even then, it's usually better expressed via an event-based/observer pattern (Book 18) or a mediator class than a direct bidirectional dependency — genuine cases where circular dependency is the "correct" design are very uncommon in well-factored code.
- Q: Does Spring's `@Lazy` annotation "fix" a circular dependency, or just delay when the error surfaces? → A: `@Lazy` on one side of the cycle can allow the application to start successfully (by deferring that bean's actual initialization until first use, breaking the immediate construction-order deadlock) — but this is still generally considered papering over the underlying design smell rather than a genuine fix, and should be used consciously, not as a default circular-dependency workaround.

Coding Exercise

L1: Reproduce the `BeanCurrentlyInCreationException` circular dependency scenario and observe the exact error message. **L2:** Fix it by extracting shared logic into a third class, as shown in this chapter. **L3:** Research `@Lazy` and explain (in your own words, in comments) why it's a deferral, not a true fix, for a circular dependency. **L4 (Interview):** Explain why Spring fails fast on constructor-injection circular dependencies, and why that's beneficial rather than merely inconvenient. **L5 (Senior):** Given a real circular dependency between two services in a legacy codebase, propose a

concrete refactoring plan to eliminate it. **L6 (Mastery):** Explain, from memory, why setter-injection "resolution" of a circular dependency is considered an anti-pattern despite technically working.

CHAPTER 7 — Configuration & Profiles

Telugu Explanation

`@Value` externalized configuration values (properties files, environment variables) ని inject చేయడానికి వాడతారు. `@Profile` bean definitions ని environment-specific గా (dev/staging/prod) activate/deactivate చేయడానికి వాడతారు — ఒకే codebase, వేర్వేరు environments లో వేర్వేరు గా behave అవుతుంది, code మార్చకుండా.

Professional English Explanation

`@Value` injects externalized configuration values (from properties files, environment variables).
`@Profile` activates/deactivates bean definitions per environment (dev/staging/prod) — the same codebase behaves differently across environments without code changes.

Java Code

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.*;
import org.springframework.stereotype.*;
import org.springframework.core.env.Environment;

interface EmailService { void send(String to, String message); }

@Service
@Profile("dev") // ONLY active when the "dev" profile
is enabled
class MockEmailService implements EmailService {
    @Override public void send(String to, String message) {
        System.out.println("[DEV MOCK] Would send to " + to + ": " + message + " (no real email
sent)");
    }
}

@Service
@Profile("prod") // ONLY active when the "prod"
profile is enabled
class RealEmailService implements EmailService {
    @Value("${smtp.host:smtp.default.com}") // externalized value, with a
DEFAULT if not set
    private String smtpHost;

    @Override public void send(String to, String message) {
        System.out.println("[PROD] Sending real email via " + smtpHost + " to " + to);
    }
}

@Configuration
@ComponentScan(basePackages = "book11")
@PropertySource("classpath:application.properties") // loads externalized key=value
config (Book 12 formalizes this)
class ProfileConfig {}

public class ConfigurationProfilesDemo {
    public static void main(String[] args) {
        var context = new AnnotationConfigApplicationContext();
        context.getEnvironment().setActiveProfiles("dev"); // activate "dev" profile
BEFORE registering config
        context.register(ProfileConfig.class);
        context.refresh();

        EmailService email = context.getBean(EmailService.class);
        email.send("ravi@example.com", "Welcome!");

        Environment env = context.getEnvironment();
        System.out.println("Active profiles: " +
java.util.Arrays.toString(env.getActiveProfiles()));
        System.out.println("Is 'prod' active? " +
env.acceptsProfiles(org.springframework.core.env.Profiles.of("prod")));
    }
}
```

Output

```
[DEV MOCK] Would send to ravi@example.com: Welcome! (no real email sent)
Active profiles: [dev]
Is 'prod' active? false
```

Internal Working

- `@Profile("dev") / @Profile("prod")` beans are conditionally registered — **only** the bean(s) matching an active profile are actually created; if neither `dev` nor `prod` were active, and both `EmailService` implementations were `@Profile`-restricted, `context.getBean>EmailService.class)` would fail entirely (no qualifying bean), which is a deliberate safety property forcing explicit environment configuration rather than silently defaulting to something possibly wrong.
- `@Value("${smtp.host:smtp.default.com}")` demonstrates the standard **placeholder-with-default** syntax — `${property.key:defaultValue}` — Spring resolves `smtp.host` from any registered property source (properties file, environment variable, command-line argument, in a well-defined precedence order), falling back to the literal default only if the property is genuinely absent from all of them.
- This chapter's manual `context.getEnvironment().setActiveProfiles("dev")` is exactly what Spring Boot's `spring.profiles.active` property (Book 12) automates — set once in configuration (or via an environment variable, ideal for containerized deployments), rather than requiring code changes to switch environments.

Real-World Example

Telugu: Real applications లో `dev` profile mock/in-memory services వాడుతుంది (fast local development కోసం), `prod` profile real external services (SMTP server, payment gateway) కి connect అవుతుంది — ఇదే code, deployment environment బట్టి వేర్వేరు నా behave అవుతుంది, ఒక్క configuration flag మార్పుడం ద్వారా.

English: Real applications use a `dev` profile with mock/in-memory services (for fast local development) and a `prod` profile connecting to real external services (SMTP server, payment gateway) — the same code behaves differently per deployment environment, controlled by a single configuration flag, exactly the pattern this chapter establishes.

Interview Answer

"`@Value` injects externalized configuration (properties files, environment variables) with optional default values via `${key:default}` syntax. `@Profile` conditionally registers bean definitions per active environment — a `dev` profile might use mock services while `prod` connects to real ones, with the same codebase adapting via configuration rather than code changes. If no bean matches the active profile for a required dependency, Spring fails to start rather than silently choosing an unintended default."

Cross Questions

- Q: What happens if a required bean is `@Profile`-restricted and no matching profile is active? →
A: The container fails to start, since no bean satisfies that dependency — a deliberate fail-fast

behavior rather than an unpredictable fallback.

- Q: What does `${smtp.host:smtp.default.com}` mean? → A: Resolve the `smtp.host` property from any configured property source; if it's genuinely not found anywhere, use `smtp.default.com` as a literal fallback default.
- Q: Can multiple profiles be active simultaneously? → A: Yes — `setActiveProfiles("dev", "debug")` (or the Spring Boot equivalent) can activate multiple profiles at once, and beans restricted to any of the active profiles are registered.

Tricky Questions

- Q: Is it possible to register a bean that's active in ALL profiles except one specific profile? → A: Yes — `@Profile("!prod")` (the `!` negation syntax) registers the bean whenever `prod` is NOT among the active profiles, useful for "everywhere except production" beans like verbose debug logging configurations.
- Q: Does `@Value`'s property resolution have a defined precedence order across multiple sources (properties file, environment variable, command-line arg)? → A: Yes — Spring defines a specific precedence order (command-line arguments generally win over environment variables, which generally win over properties files, though the exact order depends on configuration), which matters when the same property key could be set in multiple places simultaneously; Book 12 covers this precedence in full detail for Spring Boot specifically.

Coding Exercise

L1: Create two `@Profile`-restricted implementations of an interface and switch between them by changing the active profile. **L2:** Use `@Value` with a default value, testing both when the property is set and when it's absent. **L3:** Use `@Profile("!prod")` to register a bean active in every profile except production. **L4 (Interview):** Explain what happens when no bean matches the active profile for a required dependency. **L5 (Senior):** Design a profile strategy (dev/test/staging/prod) for a service with an external payment gateway dependency, specifying what each profile's implementation should do. **L6 (Mastery):** Explain, from memory, the `${key:default}` placeholder syntax and why it's useful for externalized configuration.

CHAPTER 8 — AOP Fundamentals

Telugu Explanation

AOP (Aspect-Oriented Programming) **cross-cutting concerns** (logging, security checks, transaction management — Book 09, Ch.5's manual commit/rollback ని ఇక్కడ Spring `@Transactional` గా automate చేస్తుంది) ని business logic నుండి **separate** చేయడానికి వీలు కల్పిస్తుంది. Spring AOP **proxies** (CGLIB లేదా JDK dynamic proxies) వాడి implement అవుతుంది — ఒక bean method call అయినప్పుడు, actual method కి వెళ్ళే ముందు/తర్వాత "advice" code run అవుతుంది.

Professional English Explanation

AOP (Aspect-Oriented Programming) lets **cross-cutting concerns** (logging, security checks, transaction management — Book 09, Ch.5's manual commit/rollback becomes Spring's `@Transactional` here) be **separated** from business logic. Spring AOP is implemented via **proxies** (CGLIB or JDK dynamic proxies) — when a method is called on a bean, "advice" code runs before/after/around the actual method call.

Java Code

```
import org.aspectj.lang.annotation.*;
import org.aspectj.lang.ProceedingJoinPoint;
import org.springframework.stereotype.*;
import org.springframework.context.annotation.*;

@Service
class PaymentService {
    void processPayment(double amount) {
        System.out.println("Processing payment of $" + amount);
        if (amount > 10000) throw new IllegalArgumentException("Amount exceeds limit");
    }
}

@Aspect // marks this class as containing
cross-cutting advice
@Component
class LoggingAspect {

    @Before("execution(* PaymentService.processPayment(..))") // runs BEFORE the matched
method
    void logBefore() {
        System.out.println("[AOP] Before: about to process payment");
    }

    @AfterReturning("execution(* PaymentService.processPayment(..))") // runs AFTER successful
completion (no exception)
    void logAfterSuccess() {
        System.out.println("[AOP] AfterReturning: payment processed successfully");
    }

    @AfterThrowing(pointcut = "execution(* PaymentService.processPayment(..))", throwing =
"ex")
    void logAfterException(Exception ex) {
        System.out.println("[AOP] AfterThrowing: payment failed - " + ex.getMessage());
    }

    @Around("execution(* PaymentService.processPayment(..))") // wraps the ENTIRE call -
most powerful advice type
    Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        System.out.println("[AOP] Around: starting timer");
        try {
            return joinPoint.proceed(); // ACTUALLY invokes the
real method - required!
        } finally {
            System.out.println("[AOP] Around: took " + (System.currentTimeMillis() - start) +
"ms");
        }
    }
}

@Configuration
@EnableAspectJAutoProxy // activates Spring AOP proxy
creation
@ComponentScan(basePackages = "book11")
class AopConfig {}

public class AopFundamentalsDemo {
```

```

public static void main(String[] args) {
    var context = new AnnotationConfigApplicationContext(AopConfig.class);
    PaymentService paymentService = context.getBean(PaymentService.class);

    System.out.println("Actual bean class: " + paymentService.getClass());    // NOT
    PaymentService - it's a PROXY!

    paymentService.processPayment(500.0);
    System.out.println("---");
    try {
        paymentService.processPayment(50000.0);
    } catch (IllegalArgumentException e) {
        System.out.println("Caught: " + e.getMessage());
    }
}
}

```

Output (illustrative — advice execution order can vary slightly by configuration)

```

Actual bean class: class book11.PaymentService$$SpringCGLIB$$0
[AOP] Around: starting timer
[AOP] Before: about to process payment
Processing payment of $500.0
[AOP] AfterReturning: payment processed successfully
[AOP] Around: took 2ms
---
[AOP] Around: starting timer
[AOP] Before: about to process payment
Processing payment of $50000.0
[AOP] AfterThrowing: payment failed - Amount exceeds limit
[AOP] Around: took 1ms
Caught: Amount exceeds limit

```

Internal Working

- `context.getBean(PaymentService.class).getClass()` reveals the bean is **not actually a raw `PaymentService` instance** — it's a **CGLIB-generated proxy subclass** (`PaymentService$$SpringCGLIB$$0`) that Spring created at startup, wrapping the real `PaymentService` instance; every method call on it first passes through this proxy, which invokes matching `@Aspect` advice before delegating to the real method — this is the exact mechanism behind Spring's declarative `@Transactional` (Book 09, formalizing manual commit/rollback), `@Cacheable`, and Spring Security's method-level authorization annotations (Book 14).
- The `@Around` advice's `joinPoint.proceed()` call is what **actually invokes the real, wrapped method** — forgetting to call it is a genuine, easy-to-make bug that silently prevents the real business logic from ever running at all, since `@Around` advice has full control over whether (and how many times, and with what arguments) the underlying method executes.
- Because AOP is proxy-based, it has a well-known limitation: **calling one method from another method on the SAME object, from within that object's own code (`this.otherMethod()`), bypasses the proxy entirely** — the advice on `otherMethod()` simply won't fire, since that internal call never goes through the external proxy wrapper; this "self-invocation" limitation is a frequently-tested, genuinely important gotcha in real Spring applications (commonly biting `@Transactional` and `@Cacheable` usage).

Real-World Example

Telugu: `@Transactional` (Book 09, Ch.5 manual commit/rollback ని replace చేస్తుంది), `@Cacheable`, `@Async`, Spring Security's `@PreAuthorize` (Book 14) — ఇవన్నీ AOP proxies మీదనే build అయ్యాయి. ఈ chapter అర్థం చేసుకోవడం, ఆ annotations "ఎలా పనిచేస్తాయి" అనేదాన్ని పూర్తిగా demystify చేస్తుంది. English: `@Transactional` (replacing Book 09, Ch.5's manual commit/rollback), `@Cacheable`, `@Async`, and Spring Security's `@PreAuthorize` (Book 14) are all built directly on AOP proxies — understanding this chapter fully demystifies "how" those annotations actually work, rather than treating them as unexplained magic.

Interview Answer

"AOP separates cross-cutting concerns (logging, transactions, security) from business logic using proxies — Spring wraps a bean in a CGLIB (or JDK dynamic) proxy that intercepts method calls and runs matching `@Aspect` advice (`@Before`, `@AfterReturning`, `@AfterThrowing`, `@Around`) before/after/around the real method. This is the exact mechanism behind `@Transactional`, `@Cacheable`, and `@PreAuthorize`. A critical limitation: self-invocation (a method calling another method on `this` within the same class) bypasses the proxy entirely, so advice on the internally-called method won't fire."

Cross Questions

- Q: Why does `context.getBean(PaymentService.class).getClass()` NOT return `PaymentService.class` itself when AOP is active? → A: Because the actual bean returned is a CGLIB-generated proxy subclass wrapping the real instance, not the raw class — this is fundamental to how Spring AOP intercepts method calls.
- Q: What happens if `@Around` advice forgets to call `joinPoint.proceed()`? → A: The real, underlying method is never actually invoked — the business logic silently doesn't run, a genuine and easy-to-introduce bug.
- Q: Why does self-invocation (`this.otherMethod()`) bypass AOP advice? → A: Because that call happens directly on the real object from inside its own code, never passing back through the external proxy wrapper that intercepts calls made from OUTSIDE the object.

Tricky Questions

- Q: If `PaymentService` implemented an interface, would Spring use CGLIB or a JDK dynamic proxy? → A: By default, Spring prefers JDK dynamic proxies (interface-based) when the target class implements at least one interface, and falls back to CGLIB (subclass-based) otherwise — though this default can be configured to always prefer CGLIB.
- Q: Can advice from multiple `@Aspect` classes apply to the same method, and if so, what determines their execution order? → A: Yes — multiple aspects can match the same join point; their relative ordering is determined by the `@Order` annotation (or `Ordered` interface) on the aspect classes, and is worth being explicit about when ordering matters (e.g., a security check should generally run before a logging aspect that might record sensitive data).

Coding Exercise

L1: Build a `LoggingAspect` with `@Before` and `@AfterReturning` advice around a simple service method. **L2:** Add `@Around` advice measuring method execution time, being careful to call `joinPoint.proceed()`. **L3:** Reproduce the self-invocation gotcha: call a method internally (`this.method()`) that has advice attached, and confirm the advice does NOT fire. **L4 (Interview):** Explain how Spring AOP is implemented via proxies, and what the self-invocation limitation means practically. **L5 (Senior):** Explain how `@Transactional` uses this exact AOP mechanism, connecting it to Book 09, Ch.5's manual commit/rollback pattern. **L6 (Mastery):** Explain, from memory, why forgetting `joinPoint.proceed()` in `@Around` advice silently breaks business logic, and why that's a real, documented gotcha.

CHAPTER 9 — Design Patterns Inside Spring

Telugu Explanation

Spring Framework ఇంతకుముందు మనం నేర్చుకున్న (Book 18లో లోతుగా చూడబోయే) classic design patterns meీద heavily ఆధారపడి ఉంటుంది: **Singleton** (default bean scope, Ch.3), **Factory** (`ApplicationContext` / `BeanFactory` beans create చేసే విధానం), **Proxy** (AOP, Ch.8), **Template Method** (`JdbcTemplate` — Book 09's boilerplate abstract చేస్తుంది).

Professional English Explanation

Spring Framework relies heavily on classic design patterns (covered fully in Book 18): **Singleton** (default bean scope, Ch.3), **Factory** (how `ApplicationContext` / `BeanFactory` create beans), **Proxy** (AOP, Ch.8), and **Template Method** (`JdbcTemplate` , abstracting away Book 09's JDBC boilerplate).

Java Code — Template Method in Action (Preview of `JdbcTemplate`)

```
// This chapter's hand-written TemplateMethod mirrors EXACTLY what Spring's real JdbcTemplate
does internally

abstract class JdbcOperationTemplate<T> {                                // the ALGORITHM SKELETON
    (Book 18's Template Method)

        final T execute(String sql) {                                    // fixed skeleton - NOT
overridable                                                                // overridable
            System.out.println("Opening connection (like Book 09, Ch.2)...");
            try {
                System.out.println("Executing: " + sql);
                T result = doExecute(sql);                                // the ONE varying
step - subclasses fill this in
                System.out.println("Closing connection (like Book 09's try-with-resources)...");
                return result;
            } catch (Exception e) {
                System.out.println("Handling/translating exception (like Book 09's SQLException
wrapping)...");
                throw new RuntimeException(e);
            }
        }

        protected abstract T doExecute(String sql);                    // subclasses implement
ONLY the varying part
    }

    class QueryForListTemplate extends JdbcOperationTemplate<java.util.List<String>> {
        @Override protected java.util.List<String> doExecute(String sql) {
            return java.util.List.of("row1", "row2");                    // simulated result
        }
    }

    class UpdateTemplate extends JdbcOperationTemplate<Integer> {
        @Override protected Integer doExecute(String sql) {
            return 1;                                                    // simulated rows-
affected count
        }
    }

    public class SpringDesignPatternsDemo {
        public static void main(String[] args) {
            JdbcOperationTemplate<java.util.List<String>> queryTemplate = new
QueryForListTemplate();
            System.out.println("Query result: " + queryTemplate.execute("SELECT * FROM employee"));

            System.out.println("---");

            JdbcOperationTemplate<Integer> updateTemplate = new UpdateTemplate();
            System.out.println("Rows affected: " + updateTemplate.execute("UPDATE employee SET
salary = 80000"));

            System.out.println("")

            Spring's REAL JdbcTemplate.query(sql, rowMapper) works IDENTICALLY in spirit:
            - connection open/close: handled by the template (you never write it)
            - SQLException -> DataAccessException translation: handled by the template
            - the ONE thing YOU provide: a RowMapper (like doExecute above) mapping
ResultSet rows to objects
        }
    }
}
```



```

        This is EXACTLY Book 09's manual JDBC boilerplate, abstracted via Template
Method.
        """);
    }
}

```

Output

```

Opening connection (like Book 09, Ch.2)...
Executing: SELECT * FROM employee
Closing connection (like Book 09's try-with-resources)...
Query result: [row1, row2]
---
Opening connection (like Book 09, Ch.2)...
Executing: UPDATE employee SET salary = 80000
Closing connection (like Book 09's try-with-resources)...
Rows affected: 1

Spring's REAL JdbcTemplate.query(sql, rowMapper) works IDENTICALLY in spirit:
- connection open/close: handled by the template (you never write it)
- SQLException -> DataAccessException translation: handled by the template
- the ONE thing YOU provide: a RowMapper (like doExecute above) mapping ResultSet rows to
objects
- EXACTLY Book 09's manual JDBC boilerplate, abstracted via Template Method.

```

Design Patterns Reference Table

Pattern	Where in Spring	What Book
Singleton	Default bean scope	Ch.3 (this book)
Factory Method	BeanFactory / ApplicationContext creating beans from definitions	Ch.2 (this book)
Proxy	AOP (@Transactional , @Cacheable , method interception)	Ch.8 (this book)
Template Method	JdbcTemplate , RestTemplate , TransactionTemplate — boilerplate abstracted, one varying step supplied by you	This chapter
Dependency Injection (a form of Strategy + IoC)	The entire bean-wiring mechanism	Ch.1 (this book)
Observer	ApplicationEvent / ApplicationListener	Full detail in Book 18
Decorator	Various Wrapper / Delegating classes throughout Spring	Full detail in Book 18

Internal Working

- JdbcTemplate (the real Spring class) is a direct, production-grade implementation of the Template Method pattern: it owns the fixed algorithm skeleton (open connection → execute → handle/translate exceptions → close connection, exactly mirroring Book 09's manual try-with-resources + exception-handling boilerplate) while you supply only the one varying piece —

typically a `RowMapper<T>` lambda (Book 07's functional interfaces) mapping each `ResultSet` row to your domain object.

- Recognizing these patterns **inside** a framework you already use is a genuinely valuable interview and design skill — it proves you understand *why* the framework is shaped the way it is, not just *how* to call its methods; this is precisely why senior interviews often ask "what design patterns does Spring use, and where?"
- This connects every book in the series so far: Book 02 (SOLID, especially DIP) explains *why* Spring's DI works; Book 03 (JVM) explains what a CGLIB proxy actually is at the bytecode level; Book 09 (JDBC) is exactly what `JdbcTemplate`'s Template Method automates; Book 18 will formalize all of these patterns with their full name, structure, and additional examples beyond Spring.

Real-World Example

Telugu: `JdbcTemplate.query("SELECT * FROM employee", (rs, rowNum) -> new Employee(rs.getInt("id"), rs.getString("name")))` — ఇది Book 09 అంతటిలో మీరు manual గా రాసిన connection/statement/resultset boilerplate ని పూర్తిగా తొలగిస్తుంది, మీరు కేవలం row-mapping logic మాత్రమే రాస్తారు. English: `JdbcTemplate.query("SELECT * FROM employee", (rs, rowNum) -> new Employee(rs.getInt("id"), rs.getString("name")))` completely eliminates the connection/statement/result-set boilerplate you wrote by hand throughout Book 09 — you supply only the row-mapping logic, exactly this chapter's Template Method pattern in its real, production form.

Interview Answer

"Spring relies heavily on classic design patterns: Singleton (default bean scope), Factory (how the container creates beans from definitions), Proxy (AOP — `@Transactional`, `@Cacheable`), and Template Method (`JdbcTemplate`, `RestTemplate` — the framework owns the fixed algorithm skeleton like connection management and exception translation, while you supply only the one varying step, like a row mapper). Recognizing these patterns is what separates understanding *why* Spring is shaped the way it is from just knowing *how* to call its APIs."

Cross Questions

- Q: What's the "one varying step" in `JdbcTemplate.query(sql, rowMapper)`? → A: The `RowMapper` — mapping a single `ResultSet` row to your domain object; everything else (connection handling, exception translation, resource cleanup) is the fixed template Spring provides.
- Q: How does recognizing Template Method in `JdbcTemplate` connect back to Book 09? → A: `JdbcTemplate` automates precisely the connection-open, execute, exception-handle/translate, connection-close boilerplate that Book 09 taught you to write manually via try-with-resources — it's the exact same responsibilities, just factored into a reusable template.
- Q: Is Dependency Injection itself considered a design pattern? → A: It's often discussed alongside classic GoF patterns (and relates to Strategy pattern and Factory pattern concepts), though it's more precisely a broader architectural principle/technique (implementing IoC) than one of the traditional 23 GoF patterns — Book 18 will clarify this distinction fully.

Tricky Questions

- Q: Is `RestTemplate` (Spring's HTTP client, largely superseded by `WebClient` /the `RestClient` in newer Spring versions) also a Template Method implementation? → A: Yes — it follows the same shape: Spring manages the fixed HTTP-call machinery (connection, serialization/deserialization via Jackson, Book 10 Ch.6), while you supply the varying pieces (URL, request body, response type).
- Q: Does recognizing Spring's use of Proxy (AOP) explain any limitation you learned in Ch.8? → A: Yes directly — the self-invocation limitation (Ch.8) is a direct, unavoidable consequence of AOP being proxy-based: since Proxy pattern implementations work by wrapping calls from *outside* the real object, any call originating *from inside* the real object's own code structurally cannot pass through that wrapper.

Coding Exercise

L1: Implement your own simplified Template Method for a different repeated-boilerplate scenario (e.g., resource-timing, or retry logic). **L2:** Compare your hand-written `JdbcOperationTemplate` from this chapter's demo against the real Spring `JdbcTemplate`'s actual method signatures (research via documentation). **L3:** For each pattern in the reference table, write one sentence connecting it to a Spring annotation/class you've now learned. **L4 (Interview):** Explain the Template Method pattern using `JdbcTemplate` as the concrete example. **L5 (Senior):** Explain how understanding Spring's internal design patterns helps you design your own reusable internal frameworks/libraries within a large codebase. **L6 (Mastery):** Explain, from memory, all patterns in the reference table and where each appears in Spring, without looking.

CHAPTER 10 — Mini Project: Plain Spring DI Application

Goal

Build a complete, realistic application using **plain Spring Framework** (no Spring Boot yet — that's Book 12), combining every concept in this book, to fully internalize the container mechanics before Spring Boot automates them further.

Requirements

1. **Layered architecture** (Book 10, Ch.9) with proper stereotype annotations (Ch.4): `@Repository` for an in-memory `ProductRepository`, `@Service` for `InventoryService` and `PricingService`, a top-level `OrderProcessingService` orchestrating both.
2. **Constructor injection exclusively** (Ch.5) — no field injection anywhere in the codebase.
3. At least one `@Qualifier / @Primary` scenario (Ch.5): two `DiscountPolicy` implementations (e.g., `PercentageDiscount`, `FlatDiscount`), with one marked `@Primary` and a specific injection point overriding it via `@Qualifier`.
4. **Bean lifecycle** (Ch.3): a `@PostConstruct`-initialized, singleton-scoped `InMemoryCache` component with a `@PreDestroy` cleanup logging method.
5. **Profiles** (Ch.7): a `dev` profile using an in-memory `ProductRepository`, a `prod` profile using a JDBC-backed one (reusing Book 09's DAO pattern), switchable via `@Profile`.
6. **@Value-based configuration** (Ch.7): a configurable discount threshold and tax rate loaded from a properties file, with sensible defaults.
7. **AOP logging aspect** (Ch.8): a `@Around` advice timing every method call on `OrderProcessingService`, correctly avoiding the self-invocation pitfall by ensuring all cross-service calls happen through injected beans, not internal `this.` calls.
8. Verify at startup: correct profile-based wiring, correct `@Qualifier` resolution, and proper `@PostConstruct / @PreDestroy` firing on `context.close()`.

Concepts Reinforced

Every chapter in this book — IoC/DI (Ch.1), ApplicationContext/configuration (Ch.2), bean lifecycle/scopes (Ch.3), stereotypes/component scanning (Ch.4), injection types/qualifiers (Ch.5), avoiding circular dependencies (Ch.6), profiles/externalized config (Ch.7), AOP (Ch.8), and recognizing the design patterns underneath it all (Ch.9) — applied together in one cohesive, plain-Spring application.

Stretch Goal

Add a second `@Aspect` for exception logging (`@AfterThrowing`) across the whole service layer using a package-wide pointcut expression (rather than one method), and verify both aspects apply correctly together, in a sensible order (`@Order`).

FINAL REVISION NOTES

- **IoC/DI:** object creation/wiring moves to a container; DI is the technique; requires Dependency Inversion (Book 02) already in place; Hollywood Principle.
 - **ApplicationContext:** the IoC container; `@Configuration` + `@Bean` (CGLIB-proxied for singleton guarantee), component scanning, or legacy XML.
 - **Bean lifecycle:** instantiate → inject → `@PostConstruct` → active → `@PreDestroy`; prototype beans NOT tracked for destruction after creation — a real gotcha.
 - **Stereotypes:** `@Component` / `@Service` / `@Repository` / `@Controller` — functionally identical except `@Repository`'s exception translation; single-constructor classes need no explicit `@Autowired` (Spring 4.3+).
 - **Injection types:** constructor (recommended: immutable, testable, explicit-required) > setter (optional deps) > field (discouraged). `@Primary` = default; `@Qualifier` = explicit override at injection point.
 - **Circular dependencies:** constructor injection fails fast (a feature); setter injection "resolves" it via anti-pattern partial construction; real fix = redesign, extract shared logic.
 - **Profiles/config:** `@Profile` conditionally registers beans per environment; `@Value("${key:default}")` externalizes configuration with fallback.
 - **AOP:** proxy-based (CGLIB/JDK dynamic proxy) cross-cutting concerns; `@Before` / `@AfterReturning` / `@AfterThrowing` / `@Around`; self-invocation bypasses the proxy — a critical, frequently-tested limitation.
 - **Design patterns:** Singleton (scope), Factory (bean creation), Proxy (AOP), Template Method (`JdbcTemplate`) — recognizing these is what separates "knows Spring's API" from "understands why Spring is shaped this way."
-



CHEAT SHEET

```
IoC: object creation/wiring moves to a container | DI: the technique | needs DIP (interfaces)
already in place
ApplicationContext: IoC container | @Configuration+@Bean (CGLIB-proxied, singleton-safe) |
@ComponentScan (annotation-based)
Bean lifecycle: instantiate -> inject -> @PostConstruct -> ACTIVE -> @PreDestroy (singleton
only, NOT prototype!)
Scopes: singleton(default, 1/container) | prototype(new every getBean(), NOT lifecycle-tracked
for destroy)
Stereotypes: @Component=@Service=@Controller=@Repository(+exception translation) | 1
constructor = no @Autowired needed
Injection: constructor(BEST: final, testable, explicit) > setter(optional deps) >
field(discouraged, hard to test)
@Primary = default bean | @Qualifier("name") = explicit override at injection point |
neither+ambiguous = NoUniqueBeanDefinitionException
Circular deps: constructor injection FAILS FAST (BeanCurrentlyInCreationException) - GOOD,
forces real fix
    setter injection "resolves" it = ANTI-PATTERN | real fix = extract shared logic to 3rd class
@Profile("dev"/"prod"/"!prod"): conditional bean registration per environment
@Value("${key:default}"): externalized config with fallback default
AOP: proxy-based (CGLIB/JDK) | @Before @AfterReturning @AfterThrowing @Around(needs
joinPoint.proceed())
    SELF-INVOCATION (this.method()) BYPASSES the proxy - advice won't fire - critical gotcha
Patterns in Spring: Singleton(scope) Factory(bean creation) Proxy(AOP)
TemplateMethod(JdbcTemplate/RestTemplate)
```



INTERVIEW QUESTION BANK — Spring Core

Beginner

1. What is Inversion of Control, and what is Dependency Injection?
2. What is the ApplicationContext?
3. What is the default bean scope in Spring?
4. What is the difference between @Component, @Service, and @Repository?
5. What are the three types of dependency injection?

Intermediate 6. Why is constructor injection recommended over field injection? 7. Explain the bean lifecycle, including @PostConstruct and @PreDestroy. 8. What is the difference between @Primary and @Qualifier? 9. Why does Spring fail fast on circular dependencies with constructor injection, and why is that good? 10. What is @Profile used for, and what happens if no bean matches the active profile?

Advanced 11. Explain how @Configuration's CGLIB proxying guarantees singleton behavior for inter-bean-method calls. 12. Why are prototype-scoped beans not fully lifecycle-managed, and what's the practical consequence? 13. Explain Spring AOP's proxy mechanism and the self-invocation limitation in detail. 14. Explain how JdbcTemplate implements the Template Method pattern, connecting it to Book 09's manual JDBC code. 15. Why is setter-injection "resolution" of a circular dependency considered an anti-pattern despite working?

Senior/Architect 16. Design a layered, profile-aware Spring application (dev vs prod) with proper dependency injection throughout, explaining every design choice. 17. Review a codebase using field injection everywhere — propose a migration plan and explain the concrete testability benefit. 18. Explain, end-to-end, what actually happens when `context.getBean(SomeService.class)` is called, from bean definition to fully-wired object. 19. A team reports "our @Transactional annotation isn't working" on a method called internally via `this.method()` — diagnose and explain the root cause. 20. Explain how recognizing classic design patterns (Singleton, Factory, Proxy, Template Method) inside Spring changes how you'd design your own internal framework/library.

(Full short/professional/deep-senior answer scaffolding for every question across the series lives in Book 23 — Java Interview Master Book.)



CROSS-QUESTION ENGINE — Sample Chains

Q: What is Dependency Injection? → Q: What problem does it solve? → Q: What are the three injection types? → Q: Why is constructor injection preferred? → Q: What happens with a circular dependency using constructor injection? → Q: Is that a bug or a feature?

Q: What is AOP, and how is it implemented in Spring? → Q: What are the 4 advice types? → Q: What does @Around's joinPoint.proceed() do? → Q: What happens if you forget to call it? → Q: What is the self-invocation limitation, and why does it happen? → Q: Name 2 real Spring annotations built on this mechanism.

Q: What is the default bean scope, and what are the alternatives? → Q: Does a prototype bean get @PreDestroy called? → Q: Why not? → Q: What's the risk of injecting a prototype bean into a singleton via normal field injection? → Q: How do you correctly get a fresh prototype instance on each use?



CONSOLIDATED EXERCISES (All Levels)

L1 — Beginner: Redo each chapter's L1 exercise unaided. **L2 — Intermediate:** Redo each chapter's L2/L3 exercises, explaining every Spring mechanic out loud in Telugu. **L3 — Advanced:** Build a 4-bean dependency graph with mixed injection types, qualifiers, and a `@PostConstruct` / `@PreDestroy` pair. **L4 — Interview:** Answer all 20 Interview Bank questions from memory, under 90 seconds each. **L5 — Senior:** Complete the Chapter 10 mini project fully, including the exception-logging aspect stretch goal. **L6 — Mastery:** Teach Chapters 1 (IoC/DI), 6 (circular dependencies), and 8 (AOP) out loud, from memory, using fresh examples.



ONE-DAY REVISION PLAN (≈5 hours)

Time	Focus
0:00–0:30	Ch.1: IoC/DI core idea — reproduce the before/after DI example
0:30–1:00	Ch.2: ApplicationContext/configuration — trace the CGLIB proxy singleton guarantee
1:00–1:30	Ch.3: Bean lifecycle/scopes — memorize the prototype @PreDestroy gotcha
1:30–1:45	Break
1:45–2:15	Ch.4: Stereotypes/component scanning
2:15–2:45	Ch.5: Injection types/qualifiers — the highest-density interview block
2:45–3:15	Ch.6: Circular dependencies — reproduce and fix the demo
3:15–3:45	Ch.7: Configuration/profiles
3:45–4:30	Ch.8: AOP — re-read the self-invocation limitation twice
4:30–4:50	Ch.9: Design patterns in Spring — recreate the reference table from memory
4:50–5:00	Interview Bank — answer all questions from memory



ONE-WEEK MASTER REVISION PLAN

Day	Focus
1	Ch.1–2 (IoC/DI, ApplicationContext) — set up a plain Spring project and run every example
2	Ch.3–4 (lifecycle/scopes, stereotypes) — build the prototype-vs-singleton comparison
3	Ch.5 (injection types) — refactor field injection to constructor injection with qualifiers
4	Ch.6 (circular dependencies) — reproduce, diagnose, and fix a real circular dependency
5	Ch.7 (profiles/config) — build a dev/prod profile-switching application
6	Ch.8–9 (AOP, design patterns) + Mini Project — build the full plain-Spring application
7	Full mock interview: all 20 bank questions + all cross-question chains, in Telugu and English, without notes



FINAL MASTERY CHECKLIST

- ☐ I can explain IoC and DI, and why DI requires Dependency Inversion already in place.
- ☐ I can explain how @Configuration's CGLIB proxying guarantees singleton behavior.
- ☐ I can explain the full bean lifecycle and the prototype-scope @PreDestroy gotcha.
- ☐ I can explain all 4 stereotype annotations and their one functional difference.
- ☐ I can use constructor injection with @Qualifier/@Primary correctly and explain why it's preferred.
- ☐ I can diagnose a circular dependency and redesign to fix it properly.
- ☐ I can use @Profile and @Value for environment-specific, externalized configuration.
- ☐ I can explain Spring AOP's proxy mechanism and the self-invocation limitation.
- ☐ I can identify Singleton, Factory, Proxy, and Template Method patterns inside Spring itself.
- ☐ I built the Plain Spring DI Application mini project, including the exception-logging aspect stretch goal.
- ☐ I answered the full Interview Question Bank confidently in both Telugu and English.

Once every box is checked, you are ready for [12_Spring_Boot.md](#) .